

Real-Time Lab Environment Monitoring

Real-Time Lab Environment Monitoring adalah sistem pemantauan otomatis suhu, kelembaban, dan tekanan udara laboratorium, menampilkan data secara real-time melalui web dashboard dan LabVIEW, menggantikan pengamatan termometer manual.

Proyek ini menjawab kebutuhan Laboratorium Elektronika dan Instrumentasi akan sistem pemantauan berkesinambungan dengan data historis. Sistem menggunakan arsitektur IoT tiga lapis (device, communication, application) dengan dua jalur monitoring paralel dari sumber sensor yang sama: web dashboard via MQTT dan LabVIEW via komunikasi serial.

Dua metrik utama: akurasi pembacaan sensor terhadap alat ukur referensi, dan latency data dari sensor hingga tampil di layar. Akurasi dihitung dari selisih mutlak antara nilai referensi (X_{ref}) dan nilai sensor (X_{sensor}):

$$Error = |X_{ref} - X_{sensor}|$$

Total delay end-to-end adalah akumulasi waktu dari broker MQTT hingga data terkirim ke browser via Socket.io:

$$Delay_{total} = Delay_{MQTT} + Delay_{Socket.io}$$

Alat dan Bahan

Komponen elektronik utama:

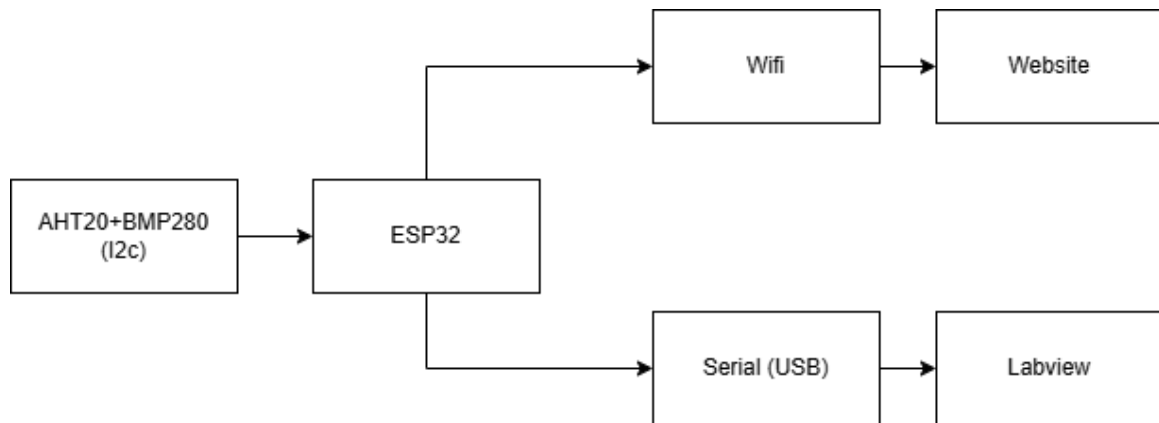
Komponen	Jumlah	Keterangan
ESP32 DevKit v1	1 unit	Mikrokontroler utama: membaca sensor via I2C, mengevaluasi alarm, mempublikasikan data ke MQTT via WiFi, dan mengirim data mentah via serial ke LabVIEW.
Sensor AHT20	1 unit	Sensor suhu & kelembaban digital I2C (alamat 0x38), akurasi $\pm 0.3^{\circ}\text{C}$ dan $\pm 2\%$ RH, tegangan operasi 2.0–5.5V, kompatibel dengan pin 3.3V ESP32.

Komponen	Jumlah	Keterangan
Sensor BMP280	1 unit	Sensor tekanan barometrik & suhu digital I2C (alamat 0x77), resolusi 0.01 hPa, rentang 300–1100 hPa, untuk memantau tekanan atmosfer ruangan.
Buzzer	1 unit	Indikator alarm, terhubung ke GPIO 2, aktif otomatis saat parameter lingkungan melewati ambang batas.
Breadboard & Jumper Wire	Secukupnya	Media prototyping untuk menghubungkan ESP32, sensor, dan buzzer tanpa PCB khusus.
Laptop (Node.js Server & LabVIEW)	1 unit	Menjalankan server backend Node.js/TypeScript (penerima data MQTT dan penyedia web dashboard) serta LabVIEW untuk monitoring via serial.
Access Point / WiFi Router	1 unit	Menyediakan jaringan lokal yang menghubungkan ESP32, broker MQTT, dan perangkat pengakses dashboard.

Desain Sistem

Arsitektur

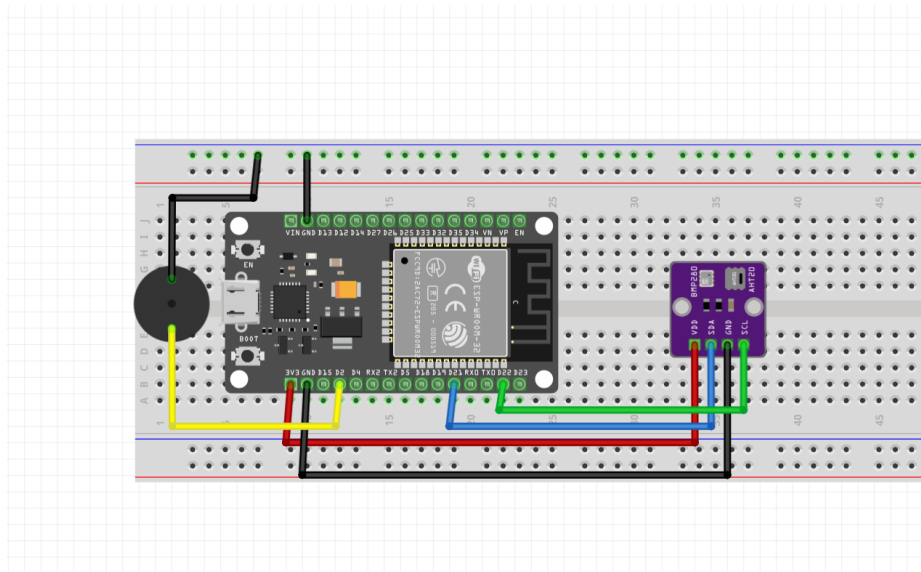
Sistem terdiri atas tiga lapis: device layer (ESP32 membaca AHT20 & BMP280 tiap 500 ms via I2C, mengevaluasi alarm, mengemas data ke JSON), communication layer (broker MQTT Mosquitto meneruskan data ke server Node.js), dan application layer (server mendorong data real-time ke browser via Socket.io serta menyimpan snapshot JSON tiap jam). ESP32 juga mengirim data yang sama via serial USB ke LabVIEW, berjalan paralel dan independen dari jalur MQTT.



Data flow diagram sistem monitoring

Wiring

AHT20 dan BMP280 terhubung ke ESP32 melalui bus I2C yang sama (shared bus): VCC ke 3.3V, GND ke GND, SDA ke GPIO 21, SCL ke GPIO 22. Pin SDO BMP280 dihubungkan ke 3.3V agar alamat I2C-nya menjadi 0x77, berbeda dari alamat default AHT20 (0x38), sehingga keduanya berbagi bus tanpa konflik. Buzzer terhubung langsung: anoda (+) ke GPIO 2, katoda (-) ke GND, tanpa komponen tambahan.



Wiring diagram rangkaian pada breadboard

Program Firmware ESP32

Inti firmware ESP32 — inisialisasi sensor & koneksi hingga logika pembacaan dan pengiriman data:

```

// SDA 21, SCL 22
#include <WiFi.h>
#include <PubSubClient.h>
#include <Wire.h>
#include <Adafruit_AHTX0.h>
#include <Adafruit_BMP280.h>

#define LED_PIN 2

const char* ssid = "BPSDM";
const char* password = "letsjumpfive";
const char* mqtt_server = "192.168.1.4";

WiFiClient espClient;
PubSubClient client(espClient);

Adafruit_AHTX0 aht;
Adafruit_BMP280 bmp;

bool aht_ok = false;
bool bmp_ok = false;

void reconnect() {
  while (!client.connected()) {
    String clientId = "ESP32_" + String(random(1000));
    if (client.connect(clientId.c_str())) {
      Serial.println("MQTT Connected");
    } else {
      Serial.print("Failed, state=");
      Serial.println(client.state());
      delay(3000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);
  delay(2000);

  Wire.begin(21, 22);
  Wire.setClock(100000);

  aht_ok = aht.begin();
  bmp_ok = bmp.begin(0x77);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }

  client.setServer(mqtt_server, 1883);
}

void loop() {
  if (!client.connected()) reconnect();
  client.loop();

  if (!aht_ok || !bmp_ok) {
    delay(1000);
    return;
  }

  // Baca sensor
  sensors_event_t humidity, temp;

```

```

aht.getEvent(&humidity, &temp);
float temperature = temp.temperature;
float hum = humidity.relative_humidity;
float pressure = bmp.readPressure() / 100.0F;

// Cek ambang batas alarm
int alarm = 0;
if (temperature > 37.5 || hum > 85.0 || pressure < 980.0 || pressure > 1100.0) {
    alarm = 1;
    digitalWrite(LED_PIN, HIGH);
} else {
    alarm = 0;
    digitalWrite(LED_PIN, LOW);
}

// Kirim ke MQTT
String payload = "{";
payload += "\"temperature\":" + String(temperature, 2) + ",";
payload += "\"humidity\":" + String(hum, 2) + ",";
payload += "\"pressure\":" + String(pressure, 2) + ",";
payload += "\"alarm\":" + String(alarm);
payload += "}";
client.publish("sensor/env", payload.c_str());

// Kirim ke serial (dibaca LabVIEW)
Serial.print(temperature, 2); Serial.print(",");
Serial.print(hum, 2); Serial.print(",");
Serial.print(pressure, 2); Serial.print(",");
Serial.println(alarm);

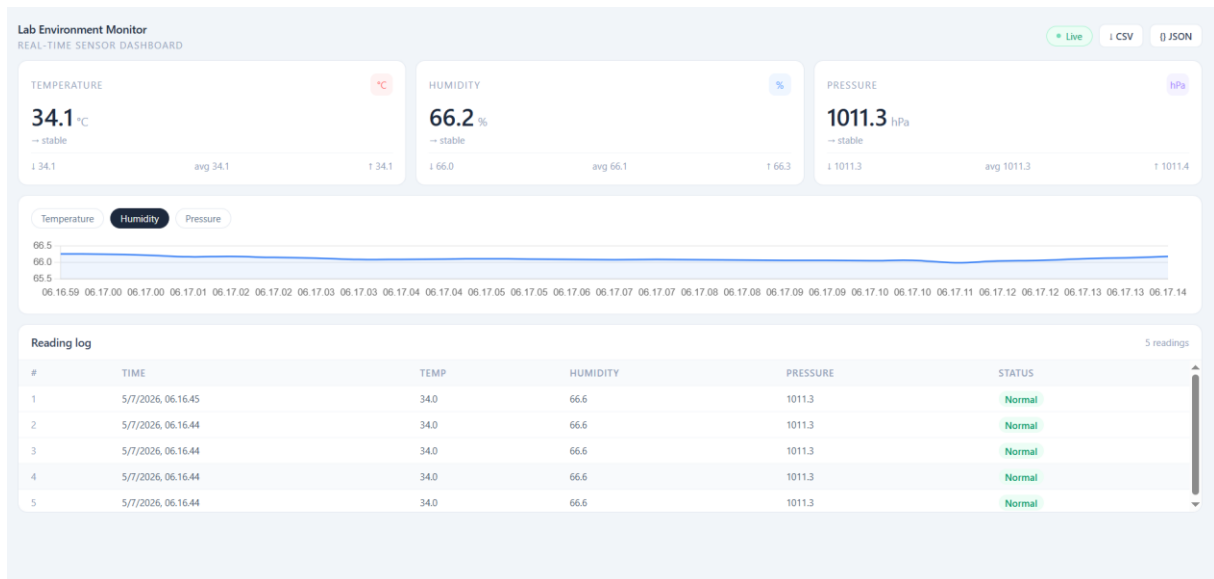
delay(500);
}

```

Program mengulang tiga langkah tiap 500 ms: membaca suhu, kelembaban, dan tekanan; mengecek ambang batas (suhu > 37.5°C, kelembaban > 85% RH, atau tekanan di luar 980–1100 hPa) untuk memicu buzzer; lalu mengirim data JSON ke topik MQTT sensor/env dan mencetaknya ke serial untuk LabVIEW.

Dashboard

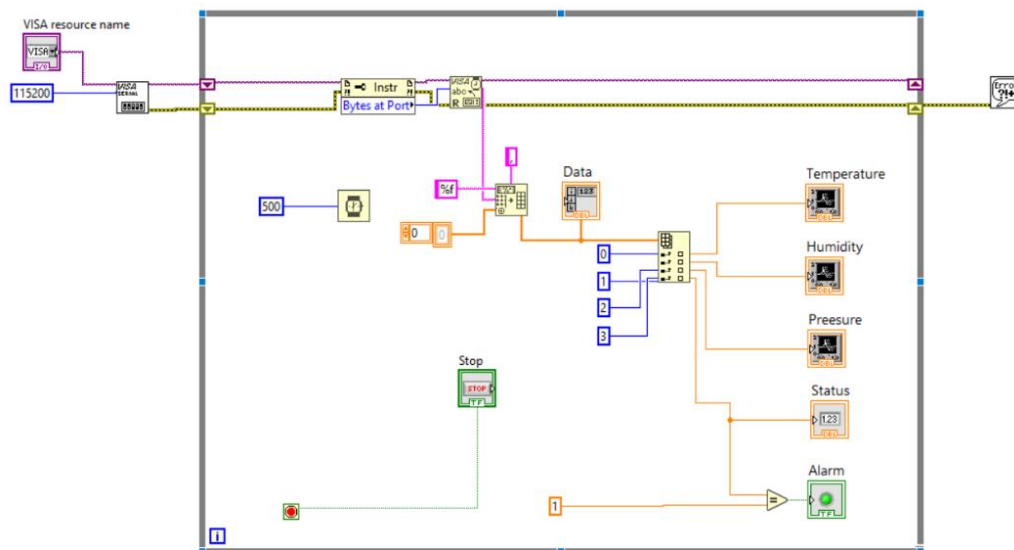
Web dashboard diakses melalui browser dari perangkat mana pun dalam jaringan lokal, tanpa instalasi tambahan. Halaman utama menampilkan suhu, kelembaban, dan tekanan real-time (update tiap 500 ms), tabel riwayat lima entri terakhir, dan ekspor data ke JSON/CSV.



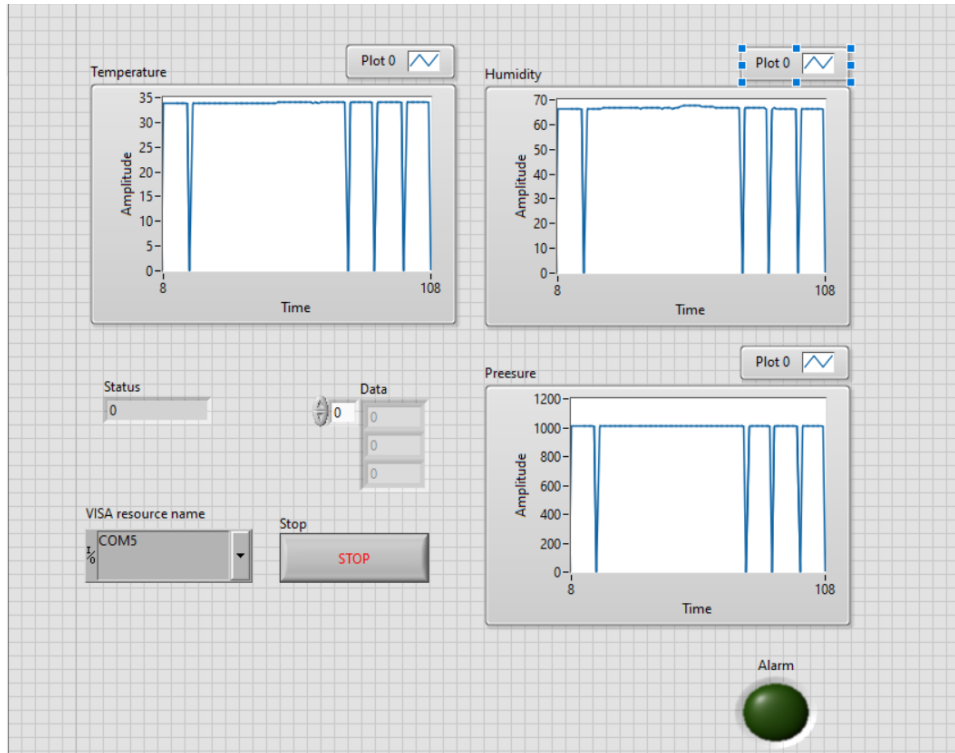
Tampilan web dashboard real-time

Antarmuka LabVIEW

Data sensor juga dapat dipantau melalui LabVIEW yang terhubung ke ESP32 via USB. Program membaca data serial tiap 500 ms, memisahkan string berbasis koma menjadi empat nilai (suhu, kelembaban, tekanan, status), lalu menampilkannya sebagai indikator dan grafik:



Block diagram program LabVIEW — membaca data serial dan menampilkannya ke indikator & grafik



Front panel LabVIEW menampilkan suhu, kelembaban, dan tekanan secara real-time

Data

Pengujian mencakup empat aspek: akurasi sensor terhadap referensi, delay komunikasi end-to-end, stabilitas pembacaan, dan fungsi alarm.

Hasil Pengujian Akurasi Sensor

Akurasi Sensor Suhu AHT20

No.	Referensi (°C)	AHT20 (°C)	Selisih (°C)
1	33.7	34.0	0.3
2	33.7	34.0	0.3
3	33.7	34.0	0.3
4	33.7	34.0	0.3
5	33.7	34.0	0.3
Rata-rata	—	—	0.3

Akurasi Sensor Kelembaban AHT20

No.	Referensi (%RH)	AHT20 (%RH)	Selisih (%RH)
1	65.0	66.6	1.6
2	65.2	66.6	1.4
3	64.8	66.5	1.7
4	65.1	66.7	1.6
5	65.0	66.6	1.6
Rata-rata	—	—	1.58

Akurasi Sensor Tekanan BMP280

No.	Referensi (hPa)	BMP280 (hPa)	Selisih (hPa)
1	1011.0	1010.4	0.6
2	1011.0	1010.6	0.4
3	1012.0	1011.3	0.7
4	1012.0	1011.5	0.5
5	1013.0	1012.4	0.6
Rata-rata	—	—	0.56

Hasil Pengujian Delay Komunikasi End-to-End

No.	Delay MQTT (ms)	Delay Socket.io (ms)	Total (ms)
1	18	22	40
2	15	19	34
3	21	25	46
4	17	21	38
5	19	23	42

No.	Delay MQTT (ms)	Delay Socket.io (ms)	Total (ms)
Rata-rata	18.0	22.0	40.0

Analisa

AHT20 menunjukkan selisih suhu rata-rata $0,3^{\circ}\text{C}$ terhadap referensi, masih dalam toleransi datasheet ($\pm 0,3^{\circ}\text{C}$). Selisih kelembaban $1,58\%$ RH terhadap DHT22 wajar mengingat toleransi kedua sensor ($\text{AHT20} \pm 2\% \text{ RH}$, $\text{DHT22} \pm 2-5\% \text{ RH}$). BMP280 menunjukkan selisih rata-rata $0,56 \text{ hPa}$ terhadap referensi eksternal, juga dalam batas spesifikasi ($\pm 1 \text{ hPa}$).

Total delay komunikasi rata-rata 40 ms ($18 \text{ ms MQTT} + 22 \text{ ms Socket.io}$), memadai untuk monitoring lingkungan yang tidak memerlukan respons kritikal. Kombinasi MQTT dan Socket.io terbukti ringan dan efisien.

Fungsi alarm bekerja sesuai rencana: buzzer aktif dalam kurang dari satu siklus loop (500 ms) setelah parameter melewati ambang batas, dan padam segera setelah kondisi normal. Kualitas sinyal WiFi dan beban jaringan lokal menjadi faktor utama yang memengaruhi variasi delay MQTT.

Kesimpulan

Sistem Real-Time Lab Environment Monitoring berhasil dibangun sebagai solusi pemantauan otomatis lingkungan laboratorium, menggantikan metode manual sebelumnya. Akurasi sensor konsisten berada dalam toleransi datasheet, dan komunikasi data via MQTT & Socket.io stabil dengan delay end-to-end rata-rata 40 ms .

Web dashboard menampilkan data secara live, dapat diakses dari perangkat mana pun dalam jaringan lokal, dengan logging per jam, ekspor JSON/CSV, dan REST API, sementara LabVIEW berfungsi sebagai jalur monitoring alternatif via serial yang independen. Pengembangan selanjutnya dapat mencakup penyimpanan time-series, notifikasi via Telegram, dan sensor kualitas udara tambahan seperti MQ-135 dan BH1750.